

Organización de Datos

Estructuras y métodos fundamentales para la gestión eficiente de información en sistemas computacionales

Organización Secuencial



Carlos Rico



Valeria A. Di Croce



Organización Secuencial

Es la organización más simple y la primera en aparecer. La única cuando los únicos dispositivos de almacenamiento permanente eran las cintas magnéticas.

Acceso Secuencial

Se basa en el acceso secuencial pero también puede optimizarse utilizando acceso relativo

Longitud Variable

La organización de registros varía según sean de longitud fija o variable

Almacenamiento

Los registros de longitud variable actualizables se almacenan en bloques

En muchos archivos transaccionales solo se hacen altas al final del archivo y nunca se actualizan registros. Ejemplos: Logs, Bitácoras, etc.



Registros Ordenados por Identificador

Ventajas

Optimización de búsquedas:

Elimina necesidad de leer todo el archivo

Procesamiento coordinado con otros archivos (Apareo)

Cortes de control con un único recorrido (Reportes)

Los registros ordenados por identificador ofrecen ventajas significativas en consultas y procesamiento, pero presentan desafíos importantes en operaciones de actualización.

Desventajas

Problemas de inserción: altas implican nuevo del archivo o deben diferirse

Bajas costosas: bajas lógicas con necesidad de re-estructuración

Bajas físicas con reconstrucción del archivo o diferidas





Registros Desordenados

01

Creación

Creación y carga sin validación de unicidad ni búsqueda de espacio libre

02

Agregar al Final

Apertura de archivo y posicionamiento al final para escribir (append)

03

Recuperación

Posicionamiento al inicio del archivo y lectura secuencial de registros hasta encontrar el buscado o llegar al final del archivo. Solo búsqueda lineal

Actualización en Registros Desordenados

1

Inserción

Se busca si el registro existe y si no existe se graba el nuevo buscando el espacio libre

2

Modificación

Se recupera el registro buscado, se actualiza, se vuelve al comienzo de la unidad recién leída y escribe la unidad modificada

3

Eliminación RRLF

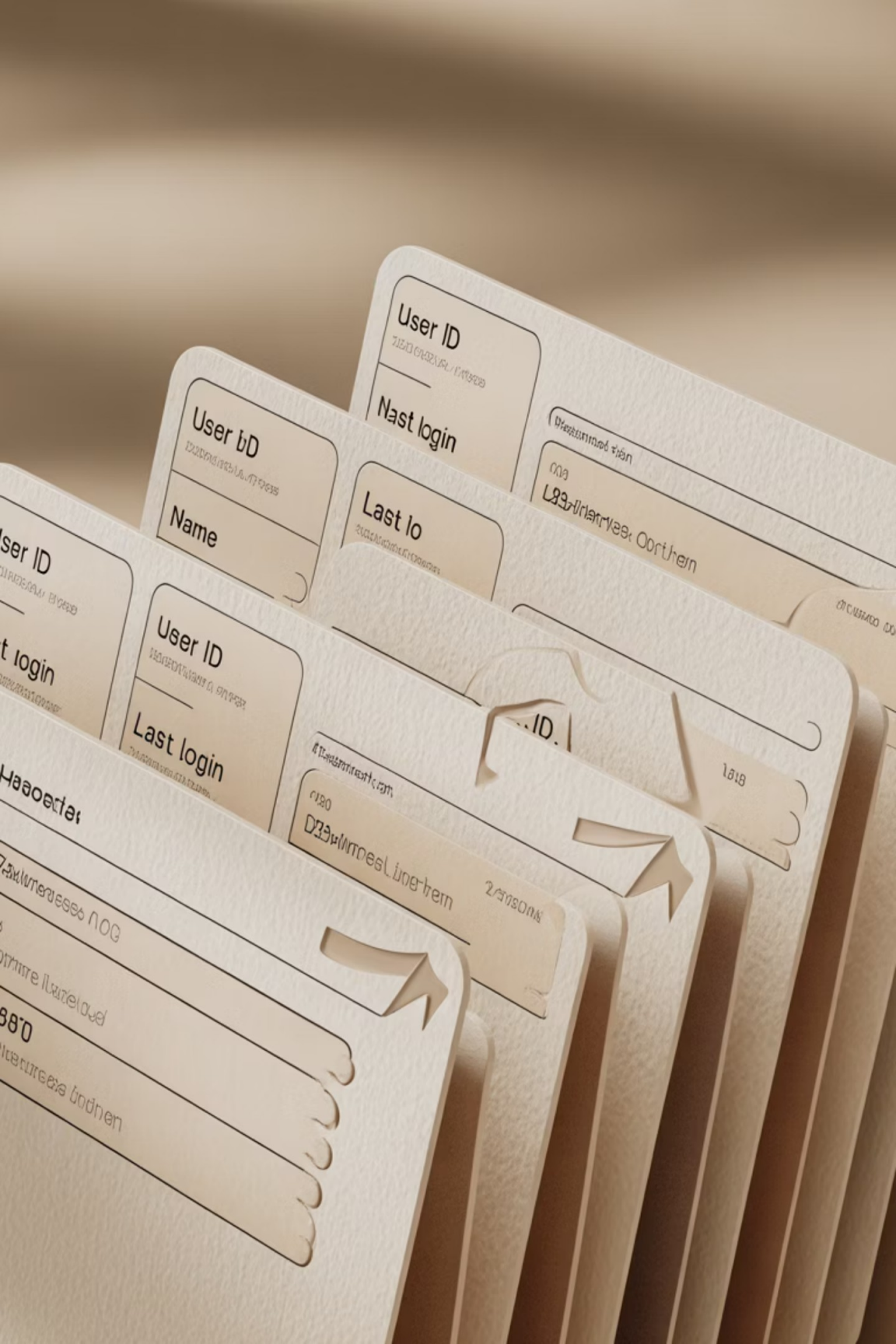
Se busca el registro a eliminar; se toma su posición; se graba el último registro del archivo en la posición a eliminar y trunca el archivo

4

Eliminación RRLV

Se compacta el bloque donde se encontró el registro; se actualiza el espacio libre del bloque y se reescribe el bloque





Manipulación de Registros Ordenados

1

Creación

Los registros a cargar ya están validados y ordenados por identificador mediante ordenamiento externo de un archivo desordenado

2

Recuperación de Registros

Consulta o recuperación unitaria mediante búsqueda lineal o binaria

3

Reportes

Cortes de control con registros ordenados por más de un campo. Se puede agrupar, contar, sumar, promediar con totales y subtotales

Actualización de Registros Ordenados

Métodos de Inserción



Inserción Directa

Crear archivo nuevo, copiar registros con identificadores menores, agregar registro nuevo, copiar resto de registros, borrar archivo viejo y renombrar



Por Novedades

Crear nuevo archivo con altas ordenadas; luego insertar por mezcla. Para recuperar un registro: consultar archivo principal y el de novedades



Modificación

Posicionar registro por identificador, leer desde el comienzo de la unidad hasta el registro y escribir. También por Apareo Maestro-Novedades-Nuevo Maestro



Eliminación en Registros Ordenados

1

Eliminación Directa

Crear archivo nuevo, copiar registros con identificadores menores al registro a eliminar, copiar el resto con identificadores mayores, borrar archivo viejo y renombrar el nuevo

2

Por Novedades

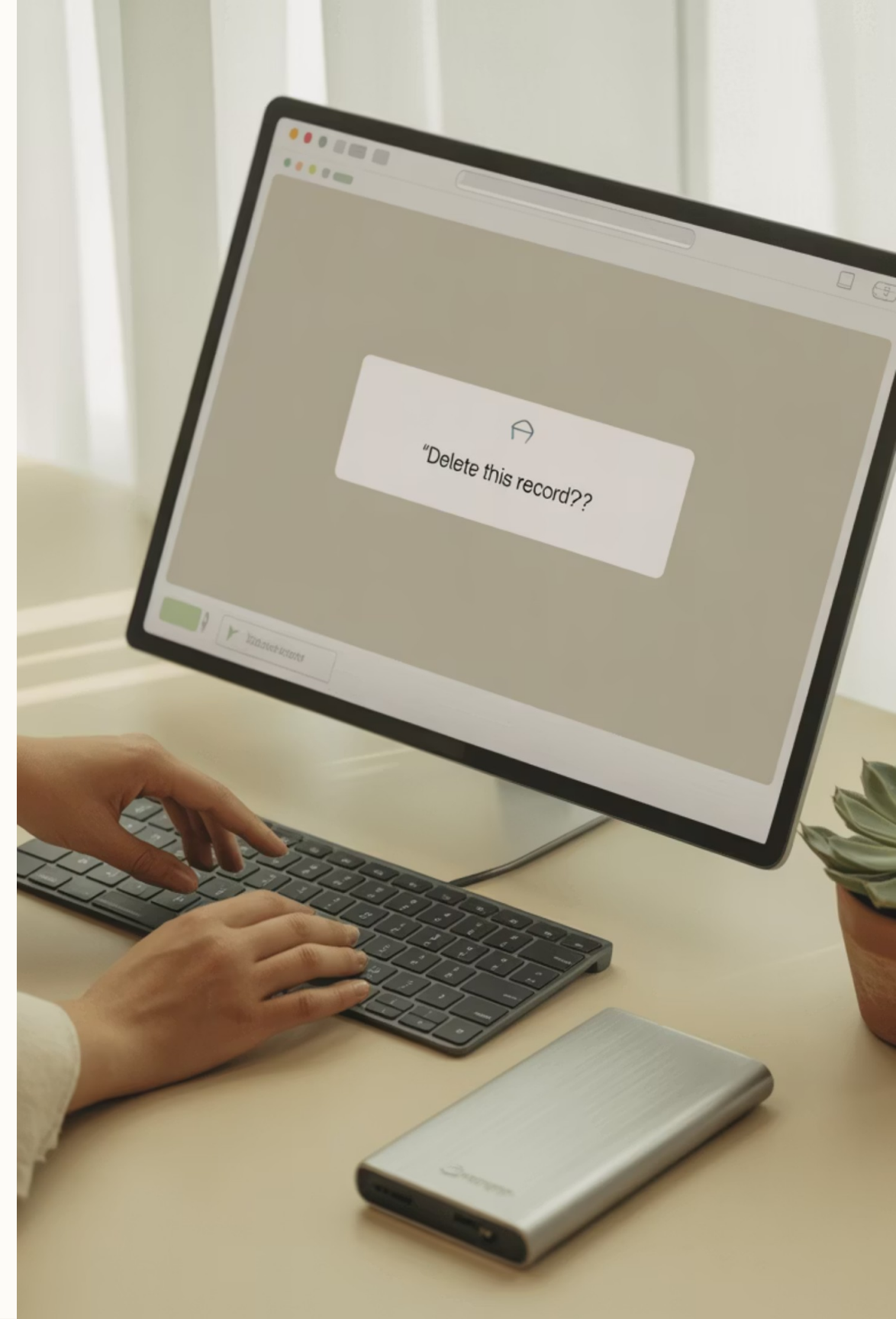
Crear nuevo archivo con los registros a eliminar; luego eliminar por mezcla

3

Por Marcado

Ubicar registro por identificador y cambiar signo del identificador, o usar campo de marcado. Requiere mantenimiento periódico para depurar o compactar

Mantenimiento: Hacer siempre copia de respaldo. En el caso de eliminación por marcado, hay que periódicamente depurar o compactar.





Conclusión

Altas:

Desordenado: $O(1)$ append.
Ordenado: costosas; directo = reconstrucción, o diferidas via novedades+apareo.

Búsqueda:

Desordenado: lineal $O(n)$.
Ordenado: binaria $O(\log n)$ (si longitud fija o índice de saltos) / lineal pero con corte temprano.

Bajas:

Desordenado: RRLF swap-último + trunc; RRLV compactación de bloque.
Ordenado: directa (reconstrucción), por novedades, o marcado + depuración.

Reportes/Cortes:

Desordenado: caros.
Ordenado: naturales y de un solo paso.



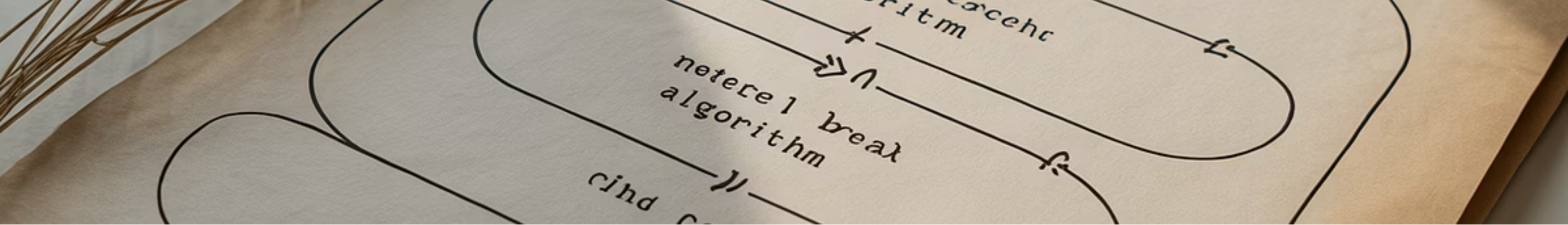
Corte de Control

Definición: Es un proceso en el cual partiendo de registros ordenados por uno o más campos, se los procesa en categorías determinadas por los criterios de ordenamiento.

En otras palabras, se procesa un conjunto ordenado de registros en subconjuntos determinados por los criterios de orden.

Aplicaciones del Corte de Control

La aplicación más común para la cual se realiza el corte de control es para generar reportes que acumulen cantidades. Se utilizan contadores, sumadores y promedios. Se informan subtotales por niveles y totales generales.



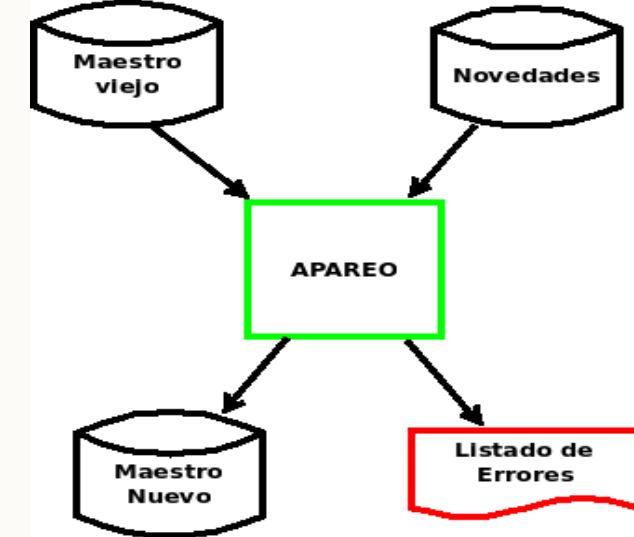
Algoritmo de Corte de Control



Este algoritmo permite procesar datos ordenados generando reportes con subtotales en diferentes niveles de agrupación.

Apareo de Archivos

A partir de dos archivos (uno principal y otro relacionado) que tienen alguna información que los enlace, generar un tercero (o una salida por pantalla), como una mezcla de los dos.

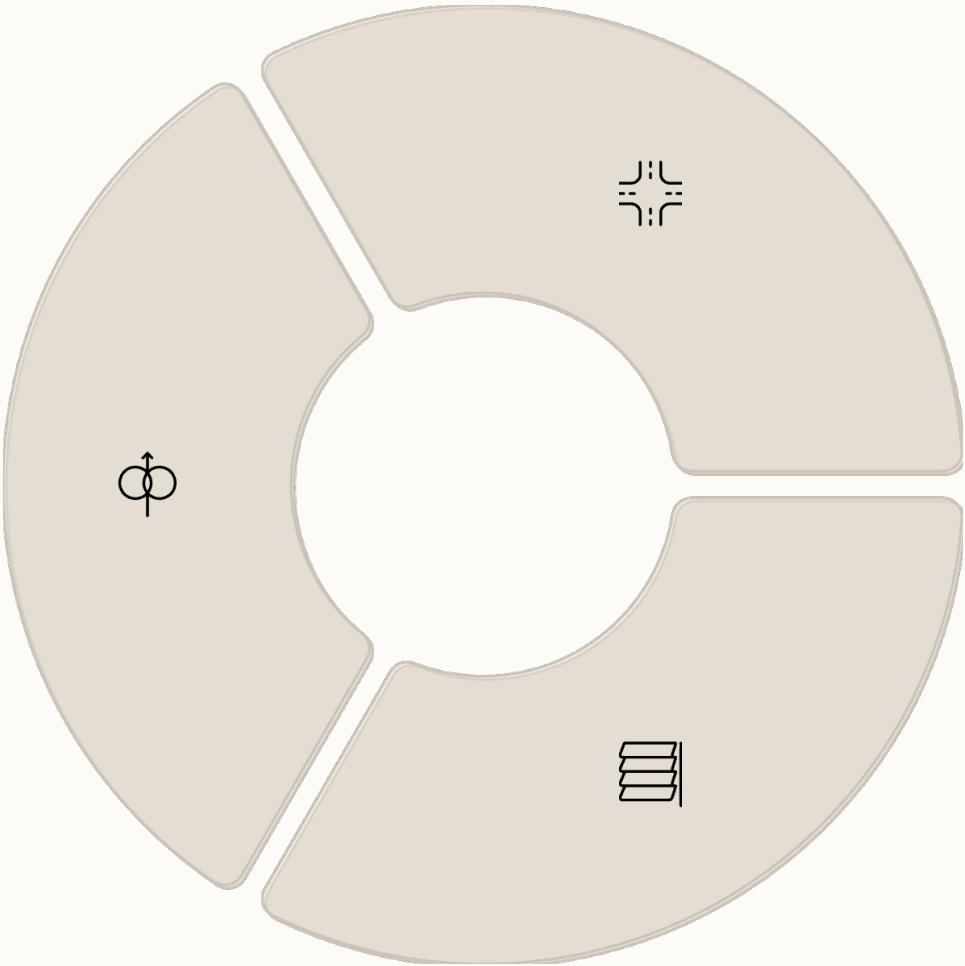


El apareo de archivos es una técnica fundamental para combinar información de múltiples fuentes de datos relacionadas, permitiendo crear nuevos conjuntos de información más completos o realizar operaciones específicas como intersecciones, diferencias y uniones.

Apareo: Intersección

A partir de dos archivos ambos ordenados (relacionados por sus identificadores)

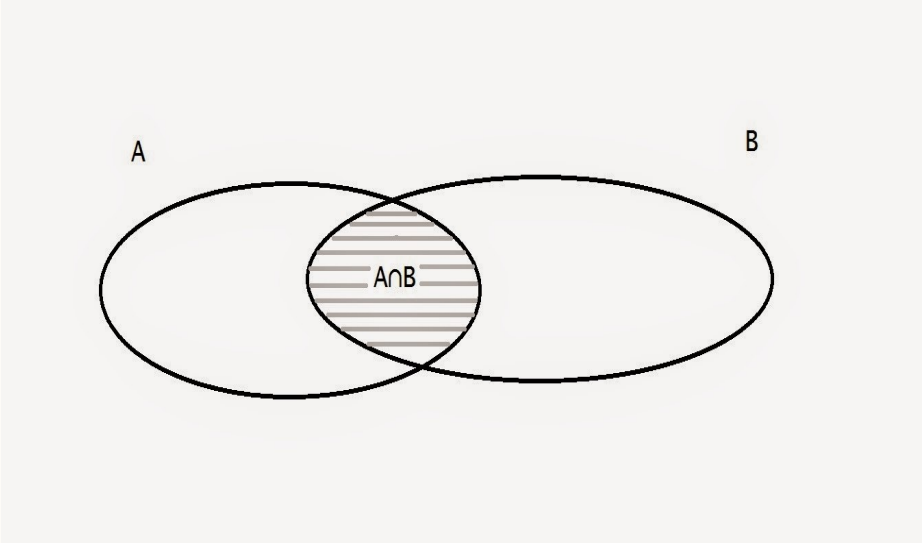
Archivo A
Datos del primer conjunto



Intersección
Solo elementos comunes

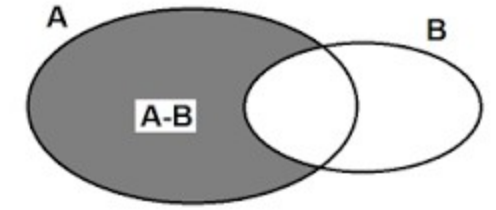
Archivo B
Datos del segundo conjunto

Usos: Quedarme con los comunes. Útil para encontrar elementos que existen en ambos archivos, como clientes activos que también son morosos.



Apareo: Diferencia

A partir de dos archivos ambos ordenados (relacionados por sus identificadores)



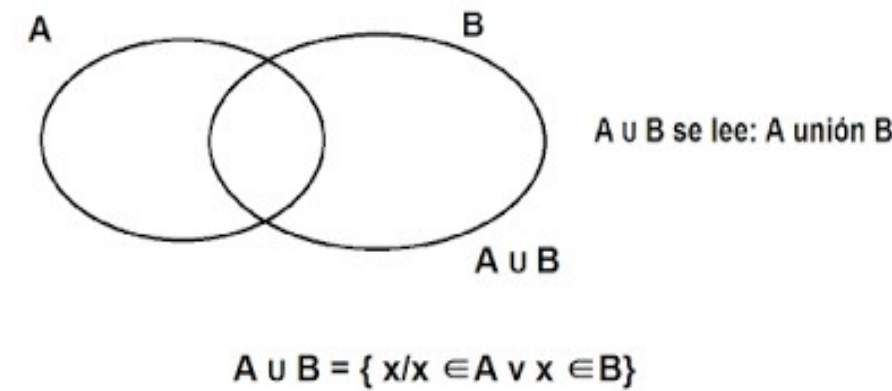
Operación de Diferencia

Obtener los registros que están en el archivo maestro pero no en el archivo de novedades.

Usos principales:

- Quedarme solo con los del Maestro menos Novedades
- Procesamiento de bajas
- Identificar elementos únicos de un conjunto

Apareo: Unión



Unión de Archivos

Combinar registros de Maestro + Novedades. Implica altas e inserciones, obteniendo el mezclado de los archivos

El apareo por unión permite crear archivos maestros actualizados incorporando todas las novedades.

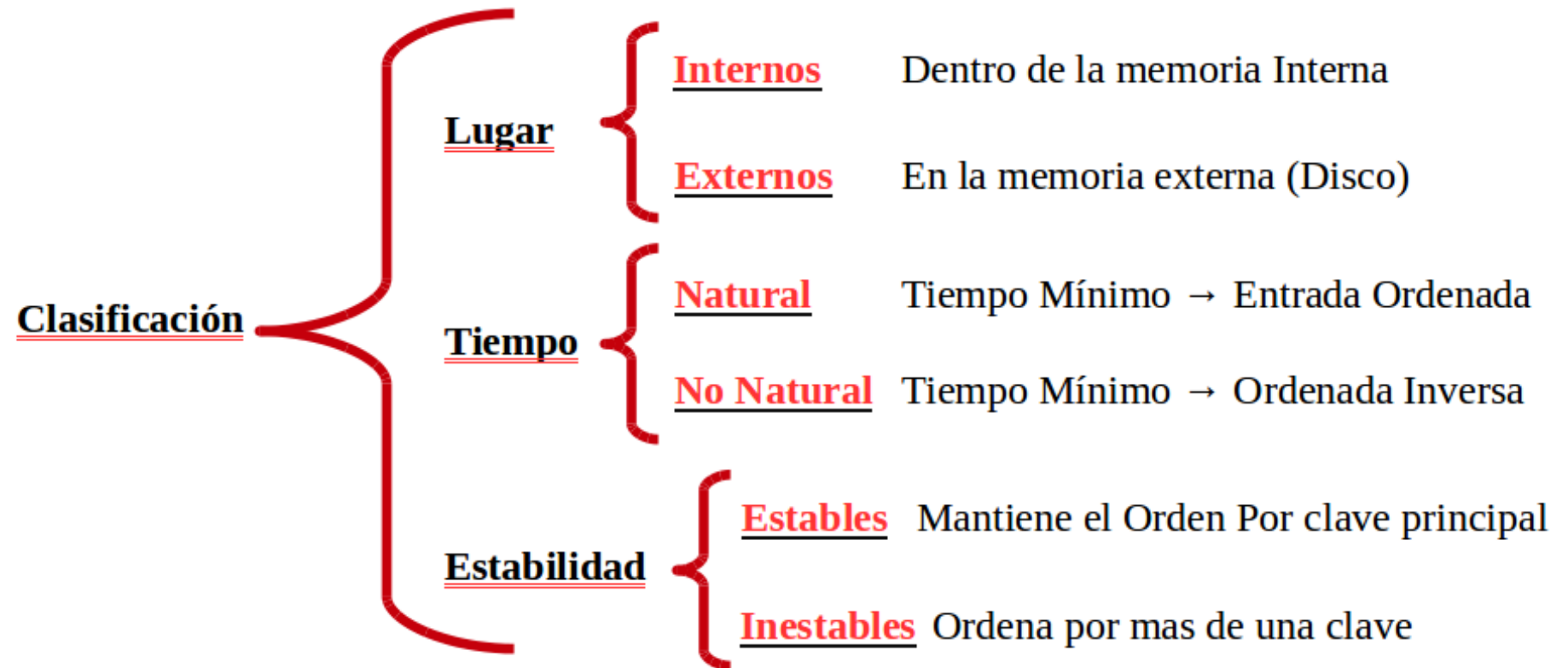
El ordenamiento es prerequisite esencial para la mayoría de operaciones eficientes de procesamiento de archivos.



Ordenamiento

Ordenamiento

Los algoritmos de búsqueda binaria, fusión, apareo y corte de control requieren tener ordenados los datos



Complejidad Computacional = $O(n)$ y Uso de Memoria = $O(n)$

Algoritmos de Ordenamiento

Algoritmo	Nombre	Complejidad	Memoria	Método	Estable
Burbuja	Bubblesort	$O(n^2)$	$O(1)$	Intercambio	Sí
Inserción	Insertion sort	$O(n^2)$	$O(1)$	Inserción	Sí
Mezcla	Merge sort	$O(n \log n)$	$O(n)$	Mezcla	Sí
Rápido	Quick Sort	$O(n \log n)$	$O(\log n)$	Dividir	No

Complejidad Computacional = $O(n)$ y Uso de Memoria = $O(n)$

Algoritmo Burbuja

Evalúa cada elemento de la lista con el siguiente intercambiándolos de posición si están en el orden equivocado

Comparación

Compara elementos adyacentes en la lista

Intercambio

Si están en orden incorrecto, los intercambia

Iteración

Repite el proceso hasta que no haya más intercambios

Aunque es simple de implementar, tiene complejidad $O(n^2)$ que lo hace ineficiente para listas grandes. Es útil principalmente con fines educativos.



Algoritmo de Inserción

Divide la lista en parte ordenada y desordenada. Evalúa cada elemento de la lista desordenada y lo inserta en la lista ordenada de a uno.



El algoritmo de inserción es eficiente para listas pequeñas y casi ordenadas. Funciona de manera similar a como las personas ordenan cartas en sus manos, insertando cada nueva carta en su posición correcta.

Métodos de Ordenamiento Avanzados



Ordenamiento por Urnas

Distribuye elementos en recipientes y luego los concatena



Ordenamiento por Mezcla

Divide recursivamente y luego mezcla las partes ordenadas



Shell Sort

Mejora inserción comparando elementos separados por intervalos decrecientes

Shell Sort fue creado por Donald Shell. Generalmente toma como intervalo inicial $n \div 2$, luego reduce los intervalos a la mitad hasta que llegue a uno. Existen variantes como Gonnet-Baeza-Yates ($N * 5 \text{ DIV } 11$) y Steven Pigeon.

Ordenamiento en Disco



El Problema: "La memoria principal no alcanza"

El ordenamiento externo se requiere cuando la información a procesar no cabe en la memoria principal (RAM). Hay que utilizar memoria externa (disco). El proceso es más lento.

900MB

Datos a Ordenar

Problema típico: ordenar 900 MB

100MB

RAM Disponible

Utilizando únicamente 100 MB de RAM

3

Archivos Mínimos

Se requieren mínimo 3 archivos para procesar

Métodos Principales

Mezcla Directa (Merge Sort): Divide en bloques, ordena en RAM, luego mezcla

Mezcla Natural: Aprovecha secuencias ya ordenadas existentes en los datos

La Mezcla Natural es más eficiente cuando existen secuencias pre-ordenadas en los datos originales.

Ordenamiento externo por Mezcla Directa

El Problema: "La memoria principal no alcanza"

Fase 1: generar “runs” (trozos ya ordenados)

1 Elegí cuánto RAM usar por tanda

- Definí RAM_MAX (p. ej., 100 MB) para cargar registros completos en memoria.

2 Leé una tanda desde el archivo fuente

- Leé secuencialmente tantos registros como quepan en RAM_MAX. Evitá “cortar” un registro entre tandas (si un registro no entra entero, lo dejás para la próxima lectura).

3 Ordená en memoria esa tanda

- Usá cualquier sort interno (quicksort, mergesort, heapsort). En este punto todo está en RAM.

4 Volcá la tanda ordenada a disco

- Escribí un archivo temporal: run_01, run_02, ... Cada run es un segmento ya ordenado.

5 Repetí

- Volvé al paso 2 hasta procesar todo el archivo. Al final tenés R runs ordenados en disco.

Convertimos un archivo gigante y desordenado en varios “Archivos ” ordenados. Ahora solo queda “coserlos” en un único ordenado.

◆ Mezcla Directa (Merge Sort)

Es el método “clásico” de ordenamiento externo cuando no todo cabe en RAM.

Pasos:

1. División en runs fijos:

- Lees bloques del tamaño que te permite la RAM (ej: 100 MB).
- Ordenás ese bloque en memoria.
- Lo volcás a disco como un *run* ordenado (`run_01` , `run_02` ...).
- Repetís hasta procesar todo el archivo.
→ Resultado: varios runs de tamaño fijo, cada uno ordenado.

2. Mezcla de runs (fase de fusión):

- Se abren k runs a la vez y se leen sus primeros buffers.
- Se selecciona siempre el menor registro de las cabeceras.
- Se escribe en un *buffer de salida*; cuando se llena, se vuelca a disco.
- Cuando un buffer de entrada se agota, se recarga desde su run hasta vaciarlo del todo.
- Se repite hasta que todos los runs se fusionan en un único archivo final ordenado.

👉 Ventaja: Controlado y predecible.

👉 Desventaja: No aprovecha si ya había partes naturalmente ordenadas.



♦ Mezcla Natural (Natural Merge Sort)

Este método busca ser “más inteligente” usando lo que ya está ordenado en el archivo.

Pasos:

1. Detectar secuencias naturales (runs):

- Lees el archivo original secuencialmente.
- Vas detectando segmentos que ya están en orden ascendente.
- Cada segmento se copia a un archivo auxiliar (se alterna entre `aux1` y `aux2`).

2. Mezclar particiones:

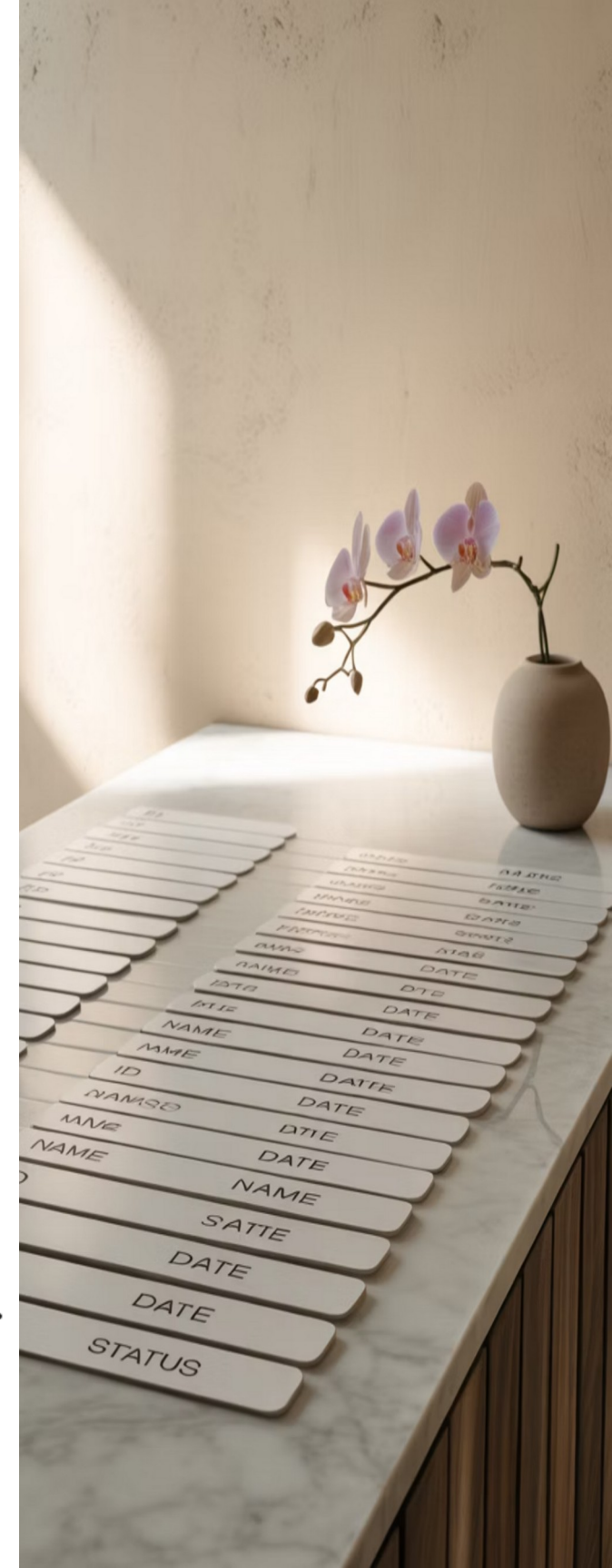
- Los runs de `aux1` y `aux2` se van apareando (como en merge sort) para formar secuencias más largas y ordenadas.
- El resultado se guarda en un nuevo archivo (o vuelve a los auxiliares).

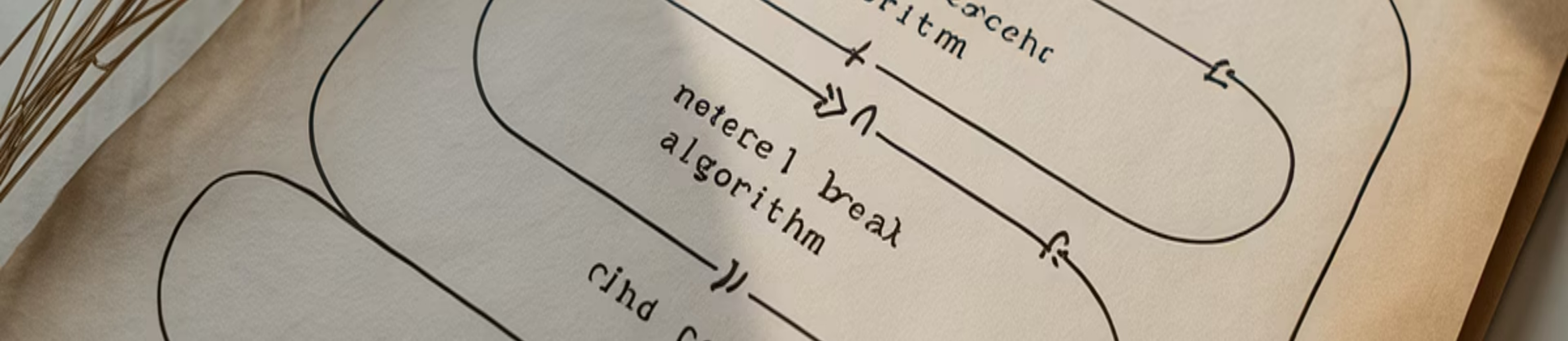
3. Repetir:

- El proceso se repite: cada vez los runs resultantes son más largos.
- Cuando finalmente hay **un solo run que abarca todo el archivo**, se logró el ordenamiento final.

👉 Ventaja: Puede necesitar **menos pasadas** si el archivo ya tiene grandes tramos ordenados.

👉 Desventaja: Se requieren al menos 2 auxiliares (a veces 3 para mayor seguridad).





Comparación rápida

Método

Cuándo usarlo

Características

Mezcla Directa

Cuando no hay garantía de orden previo

Divide en runs de tamaño fijo (RAM), luego mezcla. Eficiente pero puede requerir varias pasadas.

Mezcla Natural

Cuando se sospecha que los datos ya vienen con secuencias ordenadas (ej: logs, fechas)

Detecta runs existentes y los explota. Menos pasadas si el archivo tiene "orden natural".

1 Mezcla Directa (Merge Sort externo)

♦ EJEMPLO: Archivo ORIGEN

[10][17][05][08][09][03][22][50][04][02][20][30][11][16][19][06][21][23]

Supongamos que la RAM permite trabajar con 4 registros a la vez.

Fase 1: Generación de runs ordenados

Dividimos el archivo en bloques de 4 y los ordenamos en memoria:

Run1: [10][17][05][08] → [05][08][10][17]

Run2: [09][03][22][50] → [03][09][22][50]

Run3: [04][02][20][30] → [02][04][20][30]

Run4: [11][16][19][06] → [06][11][16][19]

Run5: [21][23] → [21][23]

Quedan en disco:

Run1: [05][08][10][17]

Run2: [03][09][22][50]

Run3: [02][04][20][30]

Run4: [06][11][16][19]

Run5: [21][23]

Fase 2: Primera mezcla Fusión de runs

Vamos fusionando de a pares:

Run1 + Run2 → [03][05][08][09][10]

[17][22][50]

Run3 + Run4 → [02][04][06][11][16]

[19][20][30]

Run5 queda igual [21][23]

Resultado intermedio:

[03][05][08][09][10][17][22][50]

[02][04][06][11][16][19][20][30]

[21][23]

1 Mezcla Directa (Merge Sort externo)

Teníamos :
[03][05][08][09][10][17][22][50]
[02][04][06][11][16][19][20][30]
[21][23]

Fase 3: Segunda mezcla

Ahora juntamos los resultados:

Mezcla [03..50] + [02..30] → [02][03][04][05][06][08][09][10][11][16][17][19][20][22][30][50]

Queda pendiente [21][23]

Resultado:

[02][03][04][05][06][08][09][10][11][16][17][19][20][22][30][50]
[21][23]

Fase 4: Mezcla final

Fusionamos los dos runs que quedan:

[02][03][04][05][06][08][09][10][11][16][17][19][20][21][22][23][30][50]

2 Mezcla Natural

♦ EJEMPLO: Archivo ORIGEN

[10][17][05][08][09][03][22][50][04][02][20][30][11][16][19][06][21][23]

Paso 1: Analizamos al Archivo ORIGEN

Dentro de este archivo ya hay subsecuencias crecientes naturales:

[10,17]; [05,08,09]; [03,22,50]; [04]; [02,20,30]; [11,16,19]; [06,21,23]

Paso 2: Distribución en auxiliares

Se copian alternadamente los runs detectados en Auxiliar1 y Auxiliar2:

Aux1: [10,17] [03,22,50] [02,20,30] [06,21,23]

Aux2: [05,08,09] [04] [11,16,19]

Paso 3: Primera fusión

Se aparean runs de Aux1 y Aux2 → nuevo archivo ORDENADO TEMPORAL:

[05][08][09][10][17][03][04][11][16][19][22][50][02][20][30][06][21][23]

Nota: Ahora ya vemos subsecuencias más largas.

2 Mezcla Natural

- ♦ Teniamos:

[05][08][09][10][17][03][04][11][16][19][22][50][02][20][30][06][21][23]

Paso 4: Segunda Fusión

Se detectan los nuevos runs y se reparten en auxiliares:

Aux1: [05][08][09][10]; [17][02][20][30]

Aux2: [03][04][11][16][19][22][50]; [06][21][23]

Ahora se aparean los runs de Aux1 y Aux2 Para dar un nuevo temporal Ordenado:

[03][04][05][08][09][10][11][16][17][19][22][50][02][06][20][21][23][30]

Paso 5: Tercera Fusión (final)

Se detectan los nuevos runs y se reparten en auxiliares:

Aux1: [03][04][05][08][09][10][11][16][17][19][22][50]

Aux2: [02][06][20][21][23][30]

Ahora se aparean los runs de Aux1 y Aux2 Para dar un nuevo Archivo Ordenado:

[02][03][04][05][06][08][09][10][11][16][17][19][20][21][22][23][30][50]

✓ Totalmente ordenado en tres pasadas.

Métodos de Ordenamiento en Disco

MÉTODO	¿CÓMO FUNCIONA?	PROS	CONTRAS
Mezcla Directa	Corta en bloques fijos y los fusiona	Fácil de entender	No aprovecha orden previo
Mezcla Natural	Usa runs ya ordenados del archivo	Menos pasadas si hay orden	Igual de costoso si desordenado
Mezcla Equilibrada	Fusiona varios runs a la vez (k-way)	Menos pasadas	Requiere más archivos
Mezcla Polifásica	Distribuye runs desbalanceados	Muy eficiente	Más compleja de implementar
Replacement Selection	Genera runs más largos con heap	Runs $\approx 2 \times$ memoria	Más difícil de programar
Radix Externo	Clasifica por dígitos o claves fijas	Muy rápido en numéricos	No sirve para todos los datos